

CENTRO UNIVERSITÁRIO FMU

Curso de Tecnologia em Análise e Desenvolvimento de Sistemas

SISTEMA DE FILA DE ATENDIMENTO BANCÁRIO

Atividade Prática Supervisionada – APS
Estrutura de Dados em Python

Vinicius Oliveira – RA: 2025104249
Rodrigo Araújo – RA: 2025108694
Mateus Vitor – RA: 2025109027
Paulo Ricardo – RA: 2025113538

Professora: Rita Cassiano

São Paulo
2026

SUMÁRIO

1 INTRODUÇÃO

2 CONTEXTUALIZAÇÃO DO PROBLEMA

3 ESTRUTURAS DE DADOS UTILIZADAS

4 SOLUÇÃO IMPLEMENTADA

5 CÓDIGO-FONTE

6 CONCLUSÃO

7 REFERÊNCIAS

1 INTRODUÇÃO

O presente trabalho foi desenvolvido no âmbito da disciplina de Estrutura de Dados em Python, como parte da Atividade Prática Supervisionada (APS). O objetivo é demonstrar a aplicação prática de estruturas de dados na resolução de um problema real, evidenciando a capacidade de abstração e escolha adequada das estruturas para cada situação.

Para isso, a equipe optou por modelar o sistema de filas de atendimento de uma agência bancária, cenário amplamente conhecido e que apresenta desafios claros que podem ser solucionados com o uso de listas e dicionários em Python.

2 CONTEXTUALIZAÇÃO DO PROBLEMA

O setor bancário é um dos mais movimentados do país, com milhares de clientes sendo atendidos diariamente em agências de todo o Brasil. Um dos principais desafios enfrentados pelas instituições financeiras é o gerenciamento eficiente das filas de atendimento, que, quando mal organizadas, geram insatisfação dos clientes, aumento do tempo de espera e sobrecarga dos atendentes.

Além disso, a legislação brasileira — por meio do Estatuto do Idoso (Lei n.º 10.741/2003) — exige atendimento prioritário para pessoas com 60 anos ou mais, gestantes e pessoas com deficiência. Diante desse cenário, a automação do processo de gerenciamento de filas por meio de estruturas de dados representa uma solução eficiente, justa e de baixo custo operacional.

3 ESTRUTURAS DE DADOS UTILIZADAS

3.1 Lista (list)

A lista foi escolhida para representar as filas de atendimento, pois é uma estrutura flexível e mutável em Python, permitindo adicionar e remover elementos

dinamicamente. Foram criadas duas listas: `filaNormal` e `filaPrioritaria`. O método `append()` insere novos clientes ao final da fila, enquanto o método `pop(0)` remove e retorna o primeiro cliente, simulando o comportamento real de uma fila — modelo FIFO (First In, First Out).

3.2 Dicionário (dict)

O dicionário foi utilizado para armazenar os dados de cada cliente no formato chave:valor, organizando informações como nome, idade e senha de forma estruturada e de fácil acesso. Essa estrutura é ideal para representar entidades com múltiplos atributos, como um cliente bancário, e permite recuperar qualquer dado diretamente pela sua chave.

4 SOLUÇÃO IMPLEMENTADA

O sistema desenvolvido apresenta um menu interativo com quatro opções de uso:

- a) Gerar Senha: cadastra o cliente e gera uma senha sequencial automaticamente. Clientes com 60 anos ou mais são direcionados à fila prioritária, recebendo senhas com sufixo 'PREF' (ex.: 1PREF, 3PREF). Clientes mais jovens recebem senhas numéricas simples (ex.: 2, 4);
- b) Mostrar fila de senhas: exibe o estado atual das duas filas — prioritária e normal;
- c) Chamar próxima senha: chama o primeiro cliente da fila prioritária; somente quando ela estiver vazia, chama da fila normal. Caso ambas estejam vazias, informa ao operador;
- d) Sair: encerra o sistema.

A detecção de prioridade é feita automaticamente com base na idade informada, sem necessidade de intervenção manual do operador, tornando o sistema mais ágil e menos suscetível a erros humanos.

5 CÓDIGO-FONTE

A seguir é apresentado o código-fonte completo do sistema desenvolvido em Python:

```

1
2 filaNormal = list()
3 filaPrioritaria = list()
4 senha = 1
5 adicionarClient = dict()
6
7 while True:
8     print("\n--- Fila do Banco ---")
9     print("1 - Gerar Senha")
10    print("2 - Mostrar fila de senhas")
11    print("3 - Chamar próxima senha")
12    print("4 - Sair")
13
14    opcao = int(input("Escolha uma opção: "))
15
16    if opcao == 1:
17        adicionarClient['nome'] = str(input("Nome: "))
18        adicionarClient['idade'] = int(input("Idade: "))
19
20        if adicionarClient['idade'] >= 60:
21            senhaPref = f"{senha}PREF"
22            filaPrioritaria.append(senhaPref)
23            print(f"Nome: {adicionarClient['nome']}, Senha: {senhaPref}")
24
25            senha += 1
26
27        else:
28            senhaNorm = senha
29            filaNormal.append(senhaNorm)
30            print(f"Nome: {adicionarClient['nome']}, Senha: {senhaNorm}")
31            senha += 1
32
33    if opcao == 2:
34        print("\n--- Fila Prioritária ---")
35        for s in filaPrioritaria:
36            print(s)
37
38        print("\n--- Fila Normal ---")
39        for s in filaNormal:
40            print(s)
41
42    if opcao == 3:
43        if filaPrioritaria:
44            chamado = filaPrioritaria.pop(0)
45            print(f"Próxima Senha: {chamado}")
46
47        elif filaNormal:
48            chamado = filaNormal.pop(0)
49            print(f"Próxima Senha: {chamado}")
50
51        else:
52            print("Não possui mais senhas.")
53
54    if opcao == 4:
55        print("Encerrando sistema.")
56        break

```

Figura 1 – Código-fonte do sistema de fila bancária. Fonte: elaborado pelos autores (2026).

6 CONCLUSÃO

O trabalho demonstrou como estruturas de dados clássicas podem ser aplicadas de forma prática na resolução de problemas do cotidiano. A lista mostrou-se a escolha ideal para o controle dinâmico das filas, pois permite inserção e remoção eficiente de elementos. O dicionário, por sua vez, organizou os dados dos clientes de forma estruturada e acessível.

A combinação dessas duas estruturas resultou em um sistema funcional, simples e eficiente para o gerenciamento de filas bancárias, respeitando a legislação de atendimento prioritário e evidenciando a importância da escolha adequada de estruturas de dados na resolução de problemas reais.

7 REFERÊNCIAS

BRASIL. **Lei n.º 10.741, de 1.º de outubro de 2003.** Dispõe sobre o Estatuto do Idoso. Diário Oficial da União, Brasília, DF, 2003.

FORBELLONE, A. L. V.; EBERSPÄCHER, H. F. **Lógica de Programação:** a construção de algoritmos e estruturas de dados. 3. ed. São Paulo: Pearson Prentice Hall, 2005.

PYTHON SOFTWARE FOUNDATION. **Python 3 Documentation.** Disponível em: <https://docs.python.org/3/>. Acesso em: mai. 2026.